RiskTrace: An Explainable ML-Based Dependency Risk Assessment and Prioritization Platform for Software Supply Chains

Project Proposal for UCS503P
Submitted to: Dr. Asif
Thapar Institute of Engineering and Technology

Project Team
Abhishek Batra (1024030463) | Shaurya Sangwan (1024030462) | Sushain Sharma (1024030439)
Batch: 3C32

August 17, 2026

1. Higher-order goal

...secondary framing

This project aims to improve software supply chain security by helping developers identify and prioritize dependency risks. The goal is to build a platform that analyzes software dependencies, detects known vulnerabilities, and produces an explainable risk score using machine learning, so that developers can see at a glance which dependencies require immediate attention rather than working through a flat, unranked vulnerability list.

2. Time-to-value

...primary heuristic

- The project will be developed in small iterations, starting with dependency collection and vulnerability detection before any modeling begins.
- ML-based risk prediction and explainability will be added only after the basic collection and matching pipeline is verified to work end-to-end.
- Success is defined by generating a dependency risk report containing vulnerability details, a priority ranking, and the reasons behind that ranking.
- CI/CD will be used for automated testing and deployment from the first iteration onward.

3. Problem Statement

Modern software applications rely on a large number of open-source libraries. Managing these dependencies and their associated security risks has become increasingly difficult as dependency trees grow deeper and less visible to the developers who rely on them.

Major challenges include:

- **Large dependency trees:** applications commonly pull in hundreds of direct and transitive dependencies, most of which were never explicitly chosen by the developer.
- **Alert overload:** existing tools generate large volumes of vulnerability warnings without indicating which ones are actually urgent.
- **Limited prioritization:** current tools rank almost entirely by raw vulnerability severity, ignoring factors like how deep a dependency sits in the tree, whether it is still maintained, and how it's actually used.

- **Manual analysis burden:** developers spend significant time manually reviewing security reports that a system could pre-triage.

Why this matters now: as software supply chains grow larger and more interconnected, efficient, explainable vulnerability prioritization is what lets developers fix the handful of critical issues first, instead of drowning in a list where everything looks equally urgent.

4. Proposed Solution

4.1 Overview

Build a web-based platform, **RiskTrace**, that analyzes a given GitHub repository and produces a prioritized, explainable dependency risk assessment. Main components:

- **Dependency Collection.** Collect dependency information using the GitHub API and deps.dev, identifying both direct and transitive dependencies.
- **Vulnerability Detection.** Match collected dependencies against OSV.dev, the GitHub Advisory Database, and the NVD to identify known vulnerabilities.
- **Risk Feature Engineering.** Generate features including vulnerability severity, dependency depth, package age, maintenance activity, and repository health.
- **ML Risk Prediction.** Primary model: XGBoost. Compared against Random Forest and Logistic Regression baselines.
- **Explainability.** SHAP is used to explain why a given dependency receives a particular risk score, rather than exposing a black-box number.
- **Dashboard.** A React + Tailwind dashboard displays the risk score, vulnerable dependencies, the reasons behind each score, and recommended actions.

4.2 Core workflow

1. User provides a GitHub repository.
2. System collects direct and transitive dependencies.
3. Dependencies are checked against vulnerability databases.
4. Risk features are generated per dependency.
5. The ML model predicts a risk score.
6. SHAP explains the prediction.
7. Results are persisted in PostgreSQL.
8. The dashboard renders the final analysis.

4.3 Operational constraints

The project focuses on practical dependency security analysis rather than building new vulnerability databases from scratch; existing, actively maintained sources (OSV.dev, GitHub Advisory Database, NVD) are used instead. The system is designed for cloud deployment.

5. Solution Approach

...engineering focus

The project combines dependency analysis, machine learning, and explainable AI.

5.1 Dependency analysis and vulnerability matching

The system collects dependency information and identifies vulnerable packages by comparing them against security databases, analyzing direct dependencies, transitive dependencies, and specific vulnerable version ranges.

5.2 Machine learning risk prediction

Dependency information is converted into security-relevant features: vulnerability severity, number of vulnerabilities, dependency depth, and package maintenance status.

Technology stack: Pandas and NumPy for data processing, scikit-learn for preprocessing, and XGBoost for prediction, compared against Random Forest and Logistic Regression.

5.3 Explainable AI layer

SHAP provides per-prediction explanations, surfacing the factors that most influenced a given risk score and the reasoning behind a high-risk classification. This is what makes the score usable for prioritization rather than just another number to trust blindly.

5.4 Web architecture

A three-layer architecture:

- **Frontend:** React + Tailwind CSS dashboard for repository input, risk visualization, and vulnerability reports.
- **Backend:** FastAPI handling repository analysis, ML inference, and API communication.
- **Database:** PostgreSQL storing repository data, dependencies, vulnerabilities, and computed risk scores.

5.5 CI/CD and fast delivery

CI/CD is used for automated testing on every change, faster and more reliable deployment, and a repeatable development workflow across the group.

6. Evaluation Criterion

...measurable and attributable

The system will be evaluated using both model performance and application-level metrics, each with a concrete target rather than an open-ended aspiration.

6.1 Primary evaluation metric: risk prediction performance

- **Target:** F1-score ≥ 0.80 and ROC-AUC ≥ 0.85 on a held-out, labeled vulnerability dataset.
- **Attribution:** measured by comparing model-predicted risk rankings against a labeled test set drawn from known-vulnerable public repositories, with results benchmarked against a naive severity-only ranking baseline.

6.2 Secondary evaluation metrics

- **Vulnerability detection accuracy:** $\geq 95\%$ of known CVEs in the test repositories correctly matched to their affected dependency.
- **Ranking quality:** top-5 flagged dependencies should contain at least 80% of the test set's confirmed critical-severity vulnerabilities.
- **Explainability coverage:** 100% of flagged dependencies must return a SHAP-based explanation alongside their score.
- **Repository analysis time:** median end-to-end scan under 60 seconds for a repository with fewer than 200 total dependencies.
- **System reliability:** $\geq 99\%$ successful scan completion rate during testing.

6.3 Validation plan

The system will be validated using public GitHub repositories with known, disclosed vulnerable dependencies, cross-referenced against labeled data from the Backstabber's Knife Collection and DataDog's malicious/vulnerable package datasets, alongside NVD- and OSV-sourced CVE records for severity ground-truth.

7. Scalability *...with foresight*

7.1 Scalable (theoretically)

The system supports scalability through modular frontend, backend, and ML components, database indexing on frequently queried fields, and a stateless, efficient API design.

7.2 Operationally deployable (practically)

The system can be deployed using cloud hosting, a managed PostgreSQL instance, and a CI/CD pipeline for staging and production releases.

8. Engine availability heuristic

The project uses existing, mature technologies rather than building infrastructure from scratch: the GitHub API and deps.dev for dependency data; OSV.dev, the GitHub Advisory Database, and NVD for vulnerability information; FastAPI for the backend; React and Tailwind CSS for the frontend; PostgreSQL for storage; XGBoost and scikit-learn for machine learning; and SHAP for explainability. This lets the team focus effort on the risk-analysis logic itself rather than reinventing any of these components.

9. Project scope and deliverables

9.1 Initial deliverable

- GitHub repository analysis and dependency extraction.
- Vulnerability matching against OSV.dev, GitHub Advisory Database, and NVD.
- PostgreSQL integration for scan persistence.

- Basic dashboard for viewing scan results.
- CI pipeline for build and test automation.

9.2 Subsequent deliverables

- ML-based risk prediction (XGBoost, with Random Forest and Logistic Regression comparisons).
- SHAP-based explanations attached to each risk score.
- Improved, prioritized recommendations for remediation.
- Expanded dashboard features (history, filtering, trend view).

10. Risks and mitigations

Data quality risk. Vulnerability information from public databases may be incomplete or delayed. *Mitigation:* cross-reference multiple independent sources (OSV.dev, GitHub Advisory Database, NVD) rather than relying on any single one.

Model reliability risk. ML-predicted risk scores may not always reflect real-world exploitability. *Mitigation:* compare multiple models against a severity-only baseline, and always pair predictions with SHAP explanations so a developer can sanity-check the reasoning, not just trust the number.

Dataset limitation risk. Labeled training data for malicious/high-risk packages is limited. *Mitigation:* combine public vulnerability datasets (NVD, OSV, Backstabber's Knife Collection, DataDog's malicious-package dataset) to broaden coverage.

Performance risk. Large repositories with deep dependency trees may increase processing time. *Mitigation:* cache resolved dependency trees, and optimize database queries and indexing for repeat scans.

11. Summary

RiskTrace is an explainable, ML-based platform for assessing and prioritizing software dependency risk. The system combines dependency analysis, multi-source vulnerability matching, XGBoost-based risk prediction, and SHAP-based explanations to help developers identify and address the security issues that matter most first. The project is scoped for practical software security, deployability, and measurable, attributable evaluation criteria within a single-semester timeline.